

E-Waste Image Classification Prototype

Process Explanation for Presentation

- This project builds a computer vision model that classifies electronic waste images into 18 categories, such as PCB, Battery, Microwave, Laptop, and Microchip-IC.
- The goal is to demonstrate a working machine learning pipeline for identifying e-waste components from optical images. This is the first step toward estimating material recovery value, because different components contain different recoverable materials.
- The prototype uses Python, TensorFlow, Keras, and transfer learning with MobileNetV2. Instead of training a computer vision model from scratch, it reuses a model that already learned general visual features from ImageNet, then adds a custom classifier for our e-waste categories.

Dataset Used

- Dataset source: Kaggle, E-Waste Image Classification Dataset (18 Classes) by harshadsgore.
- The downloaded dataset contains 27,560 images across 18 categories.
- The model training script used the Kaggle training folder as data/images. TensorFlow then automatically split those 23,960 images into 80% training and 20% validation.
- Training images: 19,168. Validation images: 4,792. Separate held-out test images: 1,800.
- The separate test folder was not used during training. It was used afterward to estimate how well the model performs on unseen data.

Class Breakdown

- Air-Conditioner: 2,314 images; Battery: 1,360; heat-sink: 1,000; Keyboard: 1,459; Laptop: 1,615; light bulbs: 1,613.

- Microchip-IC: 3,756 images; Microwave: 1,313; Mobile: 1,039; Mouse: 1,000; Passive-Component: 2,892; PCB: 1,041.

- Printer: 1,861 images; Refrigerator: 1,298; Resistor: 1,000; Television: 1,000; transistor: 1,000; Washing Machine: 999.

- Each folder name becomes a class label. For example, all images inside the PCB folder are labeled as PCB.

Step 1: Data Loading

- The script uses `keras.utils.image_dataset_from_directory()`. This function scans a folder where each category has its own subfolder.
- All images are resized to 224 x 224 pixels because MobileNetV2 expects that standard input size.
- The labels are loaded in categorical format. That means a class is represented as a one-hot vector. For example, if there are 18 classes, one image has a label like `[0, 0, 1, 0, ...]`.
- The `validation_split` parameter creates an automatic 80/20 split. The seed makes the split reproducible, so the same images are chosen each time.

Step 2: Data Augmentation

- Data augmentation creates modified versions of training images while the model trains.
- The script applies random horizontal flips, random rotations, and random zooming.
- This helps the model avoid memorizing exact images. Instead, it learns more general visual patterns, such as the shape of a circuit board, keyboard, or microwave.
- Augmentation is only active during training. Validation and prediction images are not randomly changed.

Step 3: Transfer Learning

- The model uses MobileNetV2 as a pretrained feature extractor.
- MobileNetV2 was already trained on ImageNet, a large general image dataset. That means it already learned useful low-level and mid-level visual features such as edges, corners, textures, and object parts.
- The script loads MobileNetV2 with `include_top=False`. This removes MobileNetV2's original ImageNet classifier because our classes are e-waste categories, not ImageNet categories.
- The MobileNetV2 base is frozen. Freezing means its pretrained weights are not updated during training. Only the new classification head learns.

Step 4: Classification Head

- After MobileNetV2 extracts visual features, the custom classification head converts those features into e-waste predictions.
- GlobalAveragePooling2D compresses the feature maps into a compact feature vector.
- A Dense layer with 128 neurons and ReLU activation learns combinations of the extracted features.
- Dropout randomly disables some neurons during training, which helps reduce overfitting.
- The final Dense layer has 18 neurons, one for each e-waste category. It uses Softmax activation to output probabilities that sum to 1.

Step 5: Training

- The model is compiled with the Adam optimizer and categorical cross-entropy loss.
- Adam is an adaptive optimizer. It updates trainable weights efficiently by adjusting learning behavior during training.
- Categorical cross-entropy compares the true one-hot label with the predicted Softmax probabilities.
- The model trains for 10 epochs. One epoch means the model has seen the full training set once.
- The script saves the best model checkpoint based on validation loss, so the final saved model is the best-performing epoch, not necessarily the last epoch.

Step 6: Evaluation

- After training, the model is evaluated on validation data and then separately tested on the Kaggle test folder.
- Validation accuracy: 89.86%. Validation loss: 0.2874.
- Held-out test accuracy: 89.11% on 1,800 images. Test loss: 0.3552.
- The validation and test accuracies are close, which suggests the model is generalizing reasonably well instead of only memorizing the training data.
- The training script also saves a plot of training and validation accuracy/loss over time as training_history.png.

Prediction Workflow

- After training, predict.py loads the saved Keras model and class_names.json.
- A new image is resized to 224 x 224 pixels and passed through the model.
- The model outputs 18 probabilities. The category with the highest probability is the prediction.
- Example command: `.\python313\python.exe predict.py "C:\path\to\photo.jpg"`
- A held-out PCB sample was predicted as PCB with 100.0% confidence, confirming that the saved prediction path works.

How To Explain The Architecture

- In simple terms, MobileNetV2 acts like the model's visual feature detector. It recognizes useful patterns in the image.
- The custom Dense layers act like the decision-making part. They take those visual patterns and decide which e-waste class is most likely.
- Freezing the base model makes training faster and safer because the dataset does not need to teach the model basic vision from scratch.
- Softmax turns the model output into probabilities, making the prediction interpretable.
- This is a transfer-learning architecture: pretrained visual knowledge plus a new task-specific classifier.

Limitations And Next Steps

- The dataset is useful for a prototype, but real-world performance should be tested with photos taken by the project camera under realistic lighting and backgrounds.
- Some classes may be visually similar, such as PCB, Passive-Component, Microchip-IC, Resistor, and transistor. These may require more targeted data or fine-tuning.
- The current model classifies component type. Material recovery value is not directly predicted yet. The next step is mapping each predicted category to likely recoverable materials and approximate recovery value.
- Future improvements include fine-tuning the top MobileNetV2 layers, creating a confusion matrix, adding precision/recall metrics, and collecting local e-waste photos.

Presentation Summary

- I built an end-to-end e-waste image classification prototype using TensorFlow and Keras.
- Images were organized by folder, loaded automatically, resized, and split into training and validation sets.
- I used MobileNetV2 transfer learning to avoid training a large model from scratch.
- The custom classification head uses GlobalAveragePooling2D, a ReLU Dense layer, Dropout, and a Softmax output layer.
- The model achieved about 89% accuracy on both validation and held-out test data, showing that the prototype works and is ready for real-world sample testing.